

RxJS

Реактивное программирование для фронтенд-разработчиков

От идеи потока до выбора оператора в реальном UI

events → stream → operators → UI state



Добро пожаловать в RX

Идея реактивности и разница подходов

EVENTS

TIME

STREAMS



UI — это значения во времени

Интерфейс редко ждёт один результат. Он постоянно реагирует.

**ЗАПОМНИ**

События различаются по источнику, но одинаковы по природе: это последовательности во времени.

Почему привычных инструментов недостаточно

Проблема появляется, когда источников и состояний становится много.

Callbacks

Вложенность, ручная координация и сложная отмена.

Promise

Один результат. Сам Promise не отменяется.

Event listeners

Ручная очистка и композиция обработчиков.

ЗАПОМНИ

Promise удобен для одного результата. UI требует композиции множества событий.

Что такое RxJS

Единая модель для событий, HTTP, таймеров и состояния.



ЗАПОМНИ

RxJS превращает обработку событий в читаемый конвейер.

Promise vs Observable

Похожи по назначению, различаются моделью выполнения.

Promise

- одно settled-значение
- выполняется после создания
- композиция через then / await
- связанную операцию иногда отменяет AbortController

Observable

- 0...N значений
- cold-источник обычно стартует при subscribe
- композиция через pipe + operators
- unsubscribe прекращает подписку

ЗАПОМНИ

Не делайте Promise ленивым автоматически: для отложенного создания используйте `defer(() => from(...))`.

Где живёт Observable

В Angular поток уже встроен во многие повседневные API.

HTTP + Router

HttpClient, paramMap, queryParams

Forms + Events

valueChanges, statusChanges,
fromEvent

Realtime + Time

WebSocket, interval,
пользовательские Subject

ЗАПОМНИ

Один API помогает одинаково работать с разными асинхронными источниками.

Push и Pull

Кто решает, когда появится следующее значение?

Pull

Потребитель сам запрашивает значение.

```
const value = getData();
```

Push

Источник отправляет значения, когда они появляются.

```
stream$.subscribe(render);
```

ЗАПОМНИ

Observable использует Push-модель: подписчик реагирует на новые уведомления.

Базовые элементы

Observable, Observer, Subscription и жизненный цикл

OBSERVABLE

OBSERVER

SUBSCRIPTION



Observable — описание работы

Cold Observable обычно создаёт работу отдельно для каждого подписчика.

COLD EXAMPLE

```
const stream$ = of(1, 2, 3);  
// только описание  
  
stream$.subscribe(console.log);  
// 1, 2, 3
```

Важная граница

`from(promise)` получает уже запущенный Promise.
`Subject` может отправлять значения независимо от подписчика.

Ментальная модель

Observable — контракт доставки уведомлений, а не гарантия ленивости.

ЗАПОМНИ

Ленивость — типичное свойство cold Observable, но не универсальное правило.

Жизненный цикл Observable

Контракт допускает много next, затем не более одного завершения.

next(value)

Новое значение. Может произойти 0 или много раз.

error(error)

Аварийное завершение. После него значений нет.

complete()

Успешное завершение. После него значений нет.

ЗАПОМНИ

Контракт: `next* > (error | complete)?`

Subject и BehaviorSubject

Оба являются горячими источниками и Observer одновременно.

Subject

Отправляет события активным подписчикам.

```
const submit$ = new Subject<void>();  
submit$.next();
```

BehaviorSubject

Хранит последнее значение и сразу отдаёт его новому подписчику.

```
const page$ = new BehaviorSubject(1);
```

ЗАПОМНИ

Для состояния компонента часто удобнее signal; Subject полезен как мост событий.

Subscription — это ресурс

Жизненный цикл подписки должен совпадать с жизненным циклом владельца.



RESOURCE

```
const sub = interval(1000)
  .subscribe(render);

sub.unsubscribe();
```

Предпочтительно в Angular

AsyncPipe или takeUntilDestroyed() связывают подписку с компонентом.

ЗАПОМНИ

unsubscribe прекращает подписку; отмена underlying operation зависит от источника.

Что теперь умеем

Короткая проверка перед следующим разделом

- Различать cold и hot источники
- Читать контракт next / error / complete
- Использовать Subject как мост событий
- Связывать Subscription с владельцем

Операторы создания

Как превратить значения, массивы, Promise и время в поток

OF

FROM

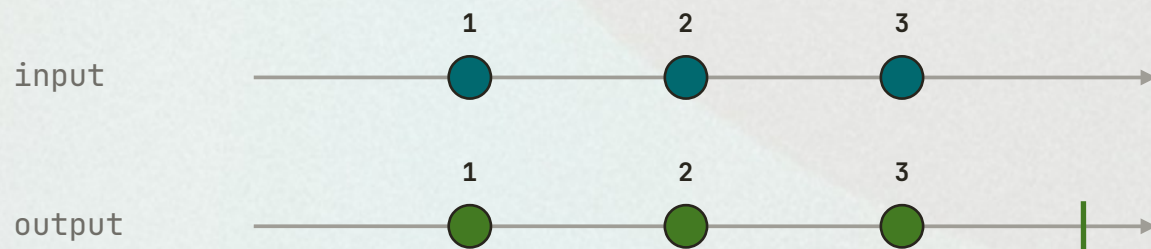
DEFER

INTERVAL



of()

Создаёт поток из готовых аргументов.



TYPESCRIPT

```
of(10, 20, 30)  
  .subscribe(console.log);
```

Когда использовать

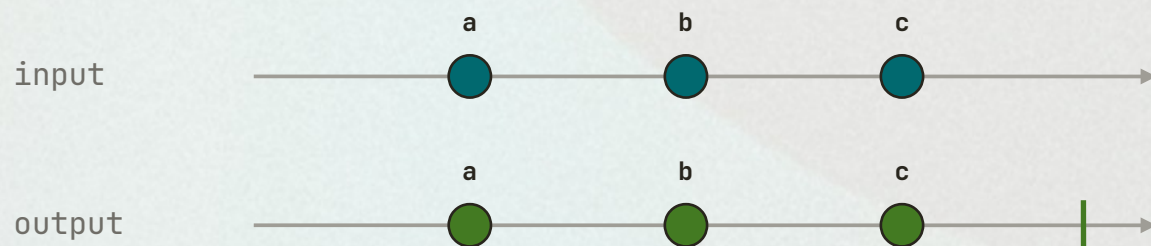
Тестовые значения, начальные данные, fallback в `catchError`.

ЗАПОМНИ

`of` сохраняет структуру каждого аргумента.

from()

Разворачивает iterable или связывает Promise с Observable.



TYPESCRIPT

```
from(['a', 'b', 'c'])  
  .subscribe(console.log);
```

Когда использовать

Массивы, iterable, Promise и сторонние асинхронные API.

ЗАПОМНИ

``from`` превращает элементы коллекции в отдельные `next`.

of([1, 2, 3]) vs from([1, 2, 3])

Одинаковый массив, разная форма потока.

of([1, 2, 3])

Один next со всем массивом:

– ([1, 2, 3]) – |

from([1, 2, 3])

Три отдельных next:

– 1 – 2 – 3 – |

ЗАПОМНИ

``of`` передаёт массив целиком. ``from`` разворачивает его элементы.

from(promise)

Результат Promise становится одним next, затем complete.

PROMISE → OBSERVABLE

```
const promise = fetch('/api/users');  
  
from(promise).subscribe({  
  next: response ⇒ render(response),  
  error: showError  
});
```

Нюанс запуска

Promise уже начал работу до вызова from(). Подписка не делает его ленивым.

Нужна ленивость?

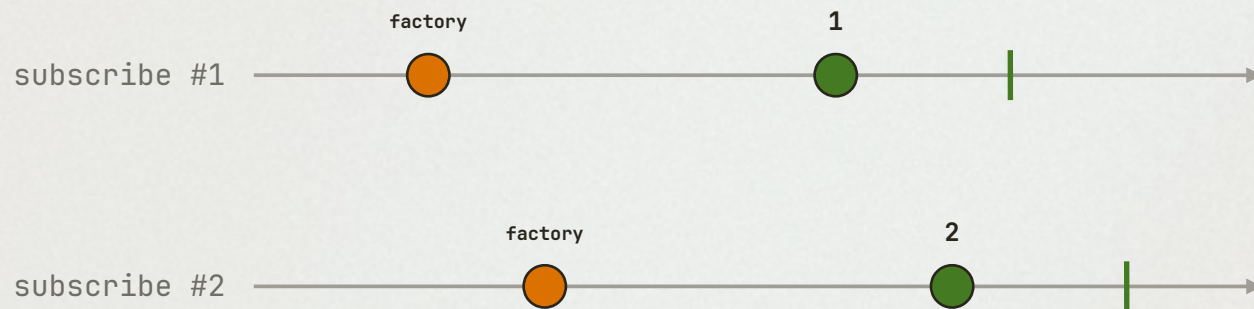
Используйте defer(() ⇒ from(fetch(...))).

ЗАПОМНИ

resolve > next + complete; reject > error.

defer()

Фабрика вызывается заново при каждой подписке.



FACTORY

```
let n = 0;  
const request$ = defer(  
  () => of(++n)  
);  
  
request$.subscribe(console.log); // 1  
request$.subscribe(console.log); // 2
```

ЗАПОМНИ

Используйте `defer`, когда параметры или сама операция должны вычисляться в момент `subscribe`.

throwError()

Создаёт поток, который сразу завершается ошибкой.



TYPESCRIPT

```
throwError(() => new Error('Boom'))  
  .subscribe({ error: console.error });
```

Когда использовать

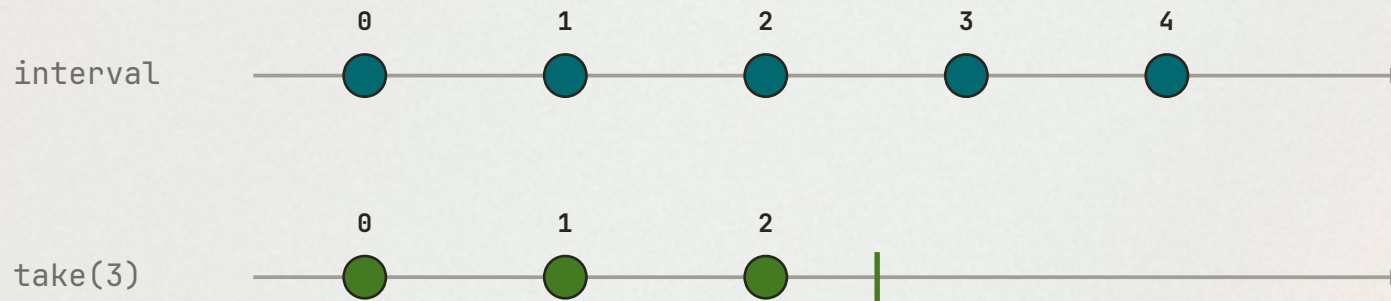
Повторный выброс ошибки из catchError после логирования.

ЗАПОМНИ

`throwError` передаёт ошибку дальше; `of(fallback)` заменяет её значением.

interval() + take()

Бесконечный таймер становится ограниченным потоком.



FINITE STREAM

```
interval(1000).pipe(  
  take(3)  
)  
.subscribe(console.log);  
// 0, 1, 2, complete
```

ЗАПОМНИ

`take(N)` завершает поток после N значений и освобождает подписку.

Что теперь умеем

Короткая проверка перед следующим разделом

● Создавать поток из значений через `of`

● Разворачивать iterable через `from`

● Откладывать создание через `defer`

● Ограничивать бесконечный поток через `take`

Pipe и операторы

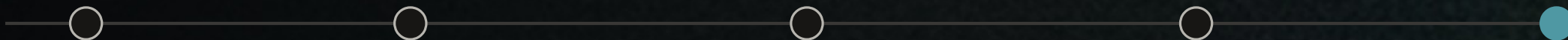
Трансформация, фильтрация и побочные эффекты

PIPE

MAP

FILTER

DEBOUNCE



pipe() — читаемый конвейер

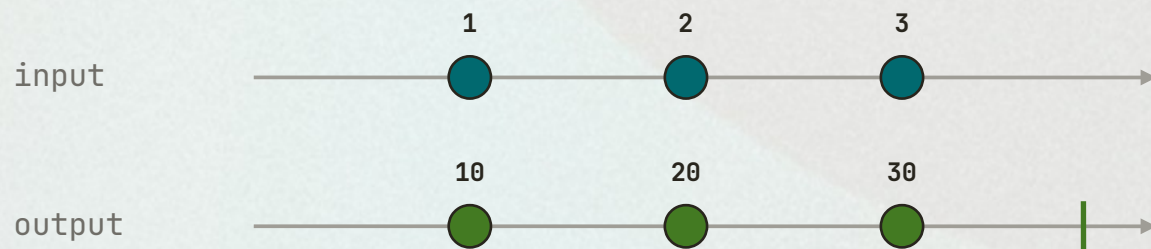
Каждый оператор получает выход предыдущего.

**ЗАПОМНИ**

Порядок операторов важен: меняется не только код, но и смысл потока.

map()

Преобразует каждое значение без изменения количества событий.



TYPESCRIPT

```
of(1, 2, 3).pipe(  
  map(x => x * 10)  
)
```

Когда использовать

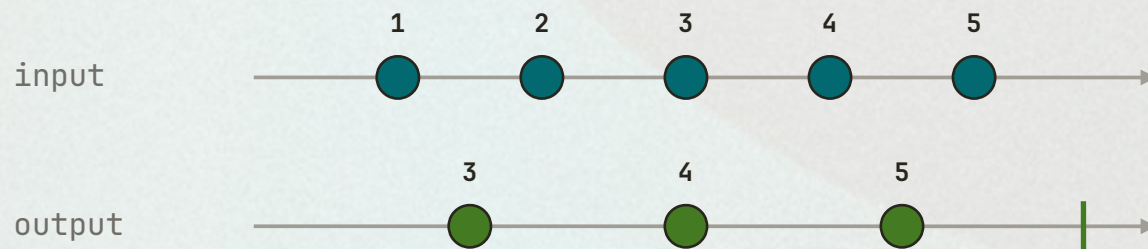
Преобразование API-модели во ViewModel.

ЗАПОМНИ

``map`` — чистая трансформация: вход > выход.

filter()

Пропускает только значения, удовлетворяющие условию.



TYPESCRIPT

```
of(1, 2, 3, 4, 5).pipe(  
  filter(x => x > 2)  
)
```

Когда использовать

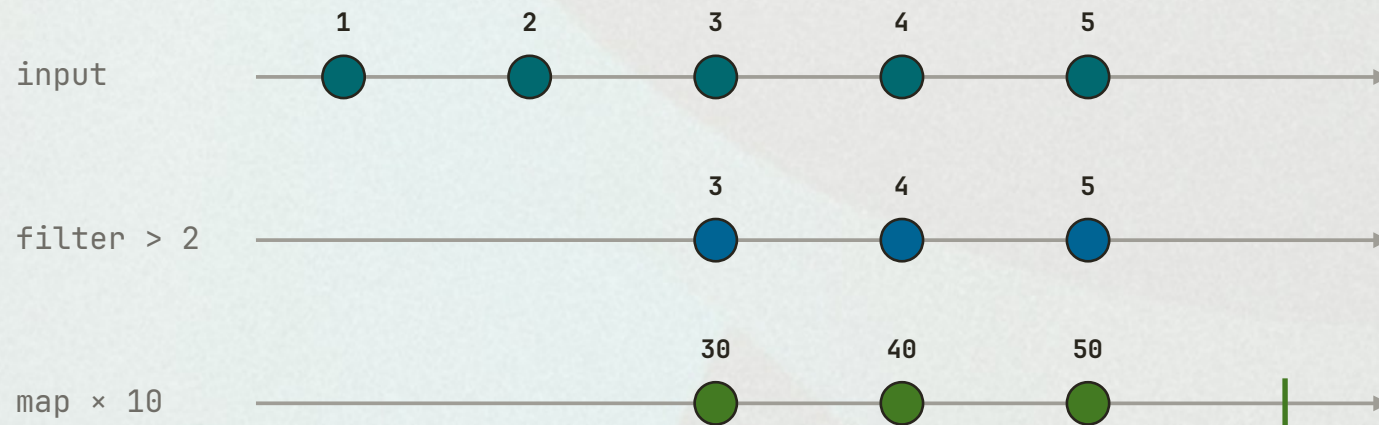
Отбрасывание невалидных или неинтересных событий.

ЗАПОМНИ

`filter` не меняет значение, а решает, пропустить ли его.

map + filter

Операторы комбинируются, а порядок определяет результат.



ORDER MATTERS

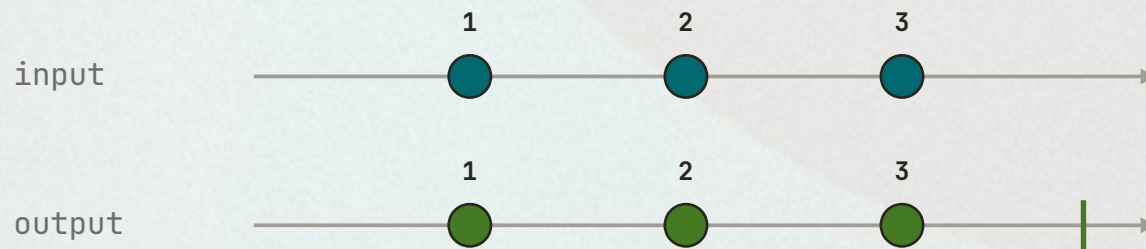
```
of(1, 2, 3, 4, 5).pipe(  
  filter(x => x > 2),  
  map(x => x * 10)  
)
```

ЗАПОМНИ

`filter > map` и `map > filter` могут давать разные результаты.

tap()

Выполняет побочное действие и не меняет значение.



TYPESCRIPT

```
source$.pipe(  
  tap(value => console.log(value)),  
  map(value => value * 10)  
)
```

Когда использовать

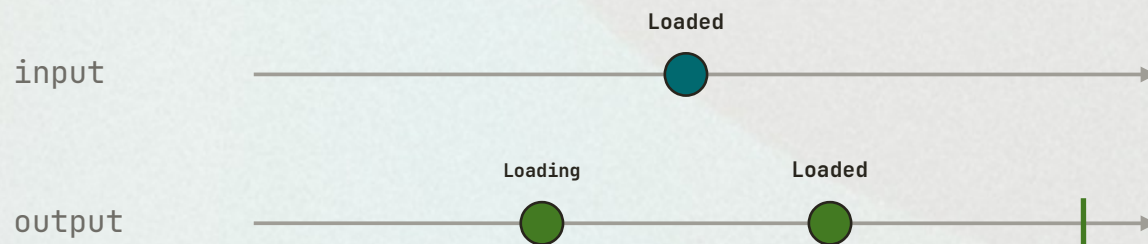
Логи, диагностика и внешние эффекты без трансформации.

ЗАПОМНИ

Если нужно изменить значение, используйте `map`, а не `tap`.

startWith()

Добавляет начальное значение перед событиями источника.



TYPESCRIPT

```
data$.pipe(  
  startWith('Loading')  
)
```

Когда использовать

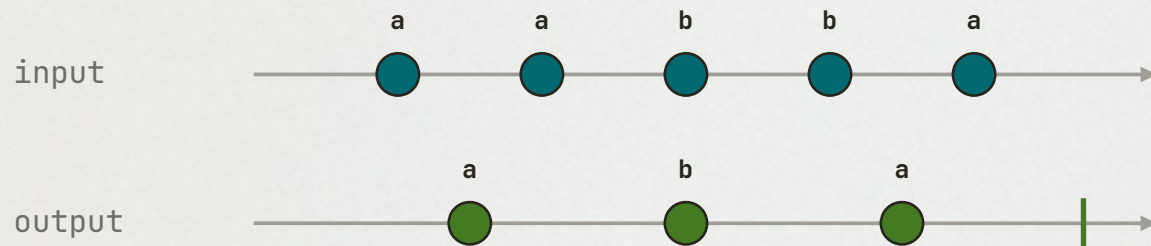
Начальное UI-состояние до первого ответа.

ЗАПОМНИ

`startWith` определяет первое значение, которое увидит подписчик.

distinctUntilChanged()

Удаляет только последовательные дубликаты.



TYPESCRIPT

```
valueChanges.pipe(  
  distinctUntilChanged()  
)
```

Когда использовать

Не запускать логику повторно, если значение не изменилось.

ЗАПОМНИ

Для объектов задайте `comparator` или сравнивайте ключ.

debounceTime()

Эмитит последнее значение после паузы во входном потоке.



LIVE SEARCH

```
search.valueChanges.pipe(  
  debounceTime(300),  
  distinctUntilChanged()  
)
```

Что происходит

Таймер перезапускается на каждом новом событии. Выходит только последнее значение после тишины.

ЗАПОМНИ

``debounceTime`` = «подожди, пока пользователь перестанет печатать».

Что теперь умеем

Короткая проверка перед следующим разделом

- Строить конвейер через pipe
- Отбрасывать события через filter

- Трансформировать через map
- Управлять частотой UI-событий

Higher-order mapping

Четыре стратегии для конкурирующих внутренних потоков

SWITCHMAP

MERGEMAP

CONCATMAP

EXHAUSTMAP



Событие запускает новый Observable

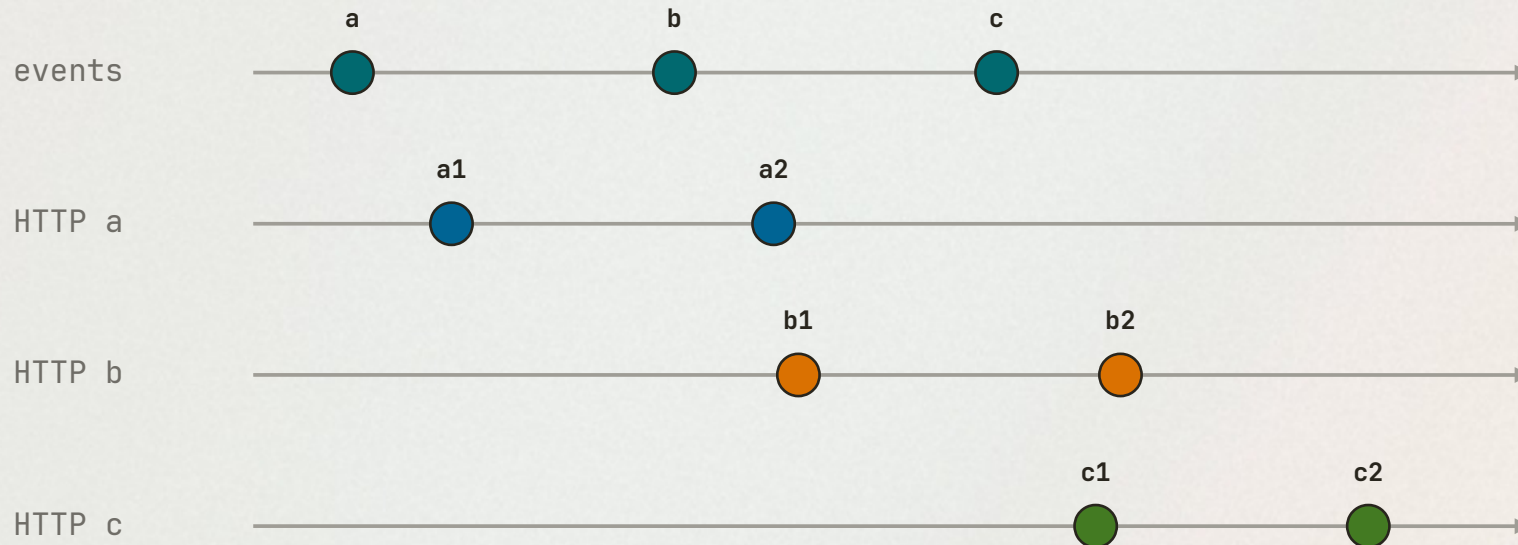
Значение внешнего потока может создавать HTTP-запрос.

**ЗАПОМНИ**

Higher-order operator управляет подписками на внутренние Observable.

Что делать, если событий несколько?

Новый запрос начинается, пока предыдущий ещё выполняется.



Четыре стратегии

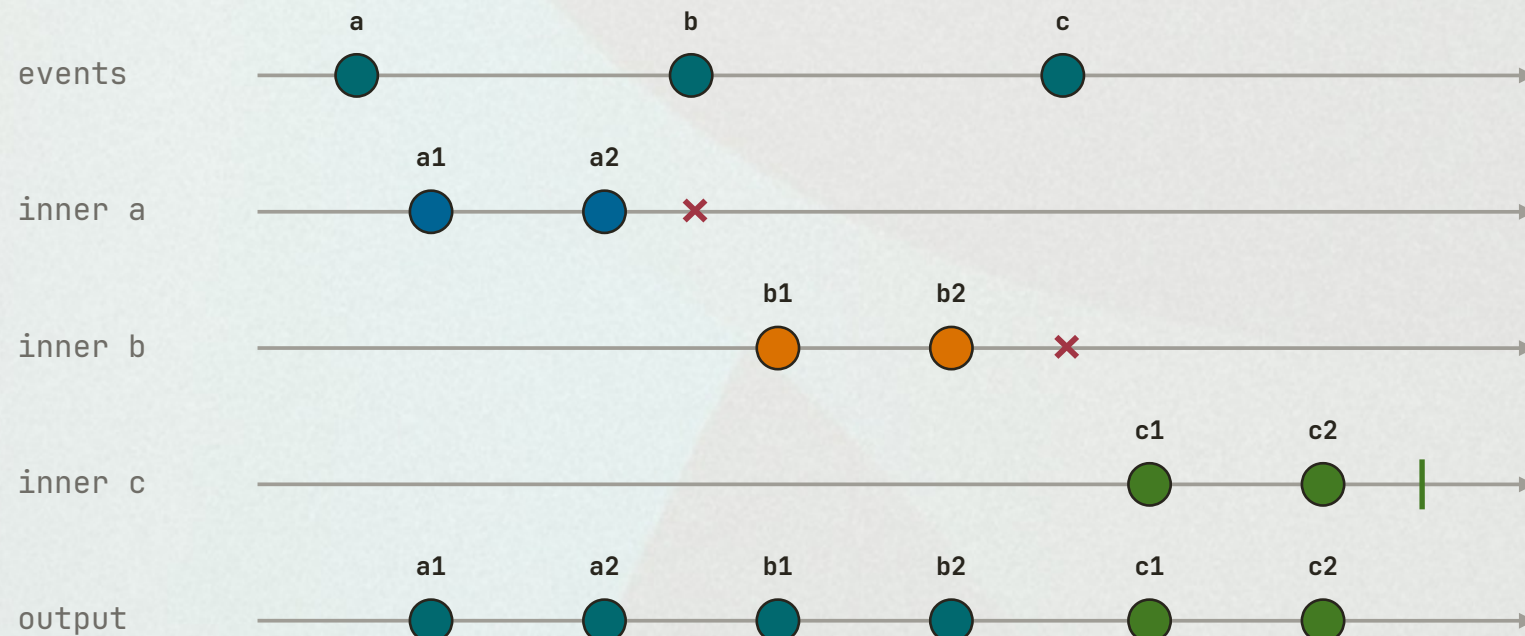
- Отменить
- Параллелить
- Поставить в очередь
- Игнорировать новое

ЗАПОМНИ

Выбор оператора — это решение о конкуренции, порядке и потере событий.

switchMap — отменить прошлое

Новый input переключает подписку на новый внутренний поток.



Лучше всего

Живой поиск
Навигация
Устаевающие запросы

Опасно

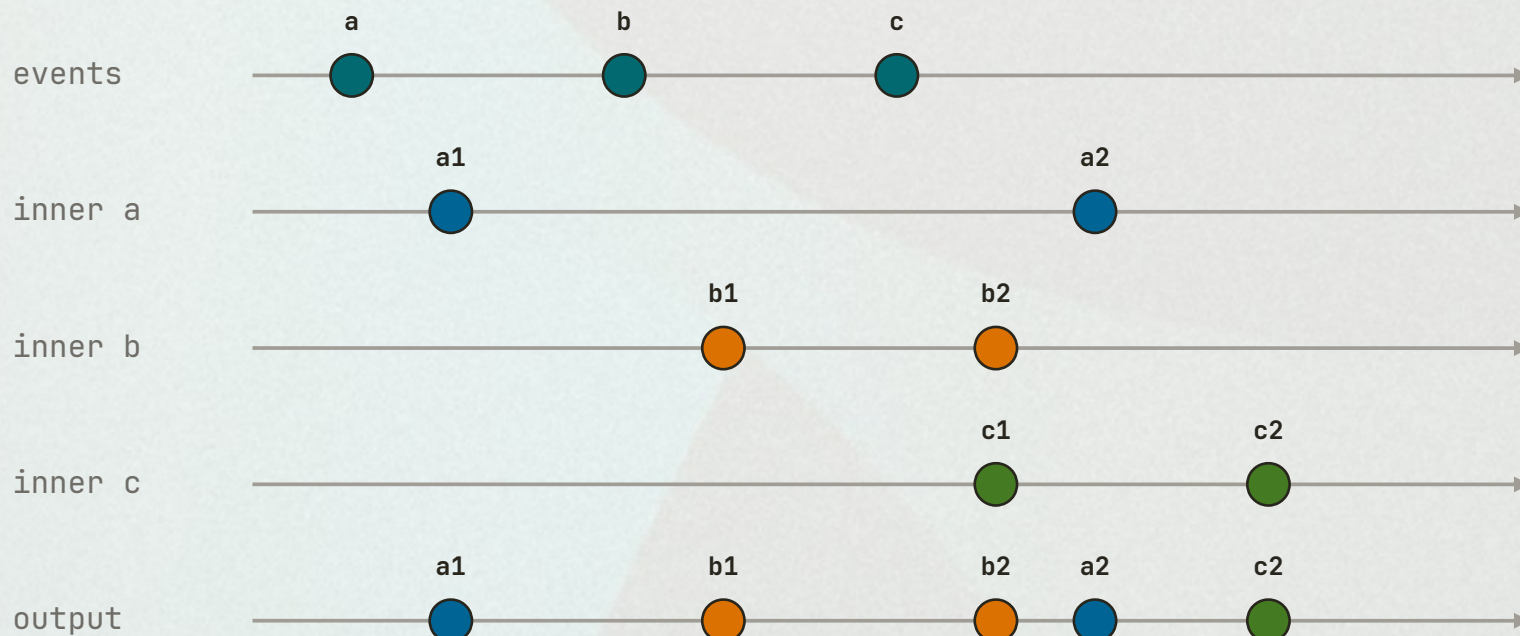
Сохранение данных: новый input может отменить важный POST.

ЗАПОМНИ

`switchMap` = «забуди прошлое, работай с актуальным».

mergeMap — выполнять параллельно

Все внутренние потоки активны одновременно.



Лучше всего

Независимые загрузки
Логи
Фоновые действия

Опасно

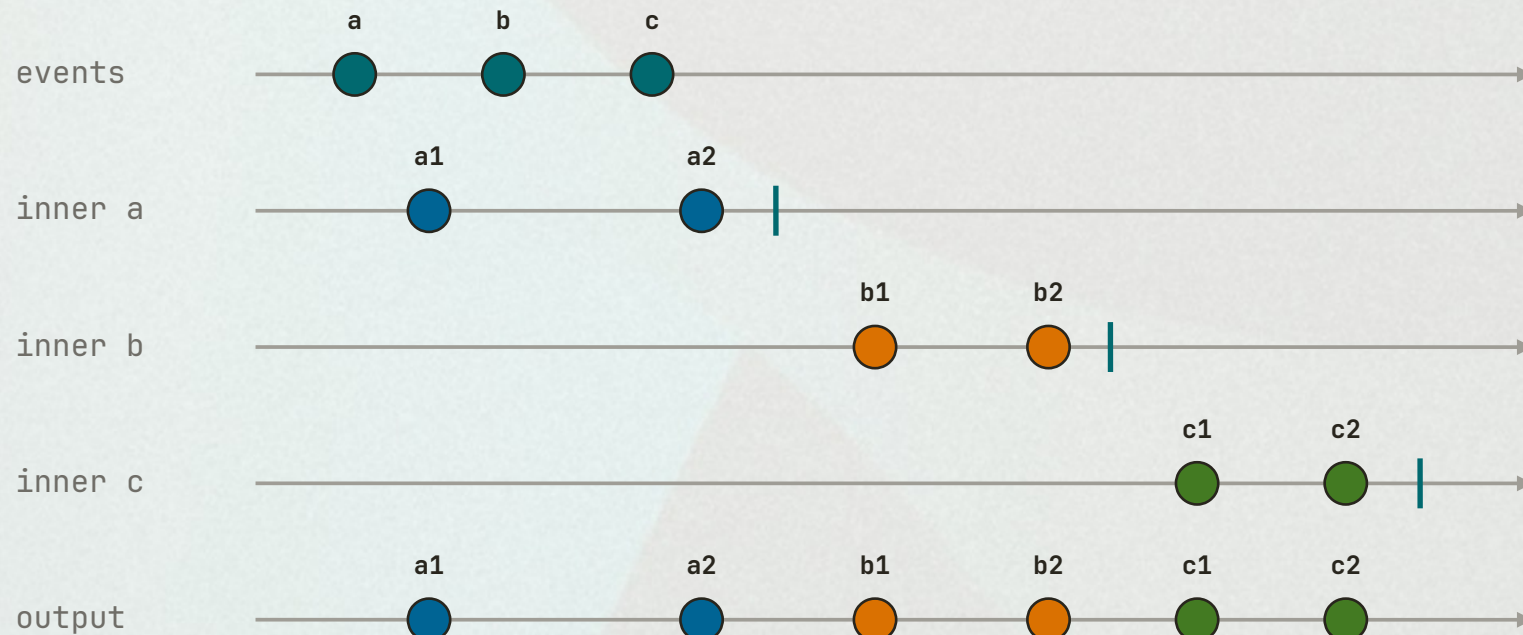
Для поиска ответы могут прийти
не в том порядке.

ЗАПОМНИ

`mergeMap` = «запусти всё; результаты приходят по готовности».

concatMap — поставить в очередь

Следующая работа ждёт завершения предыдущей.



Лучше всего

Последовательные сохранения
Команды с важным порядком

Опасно

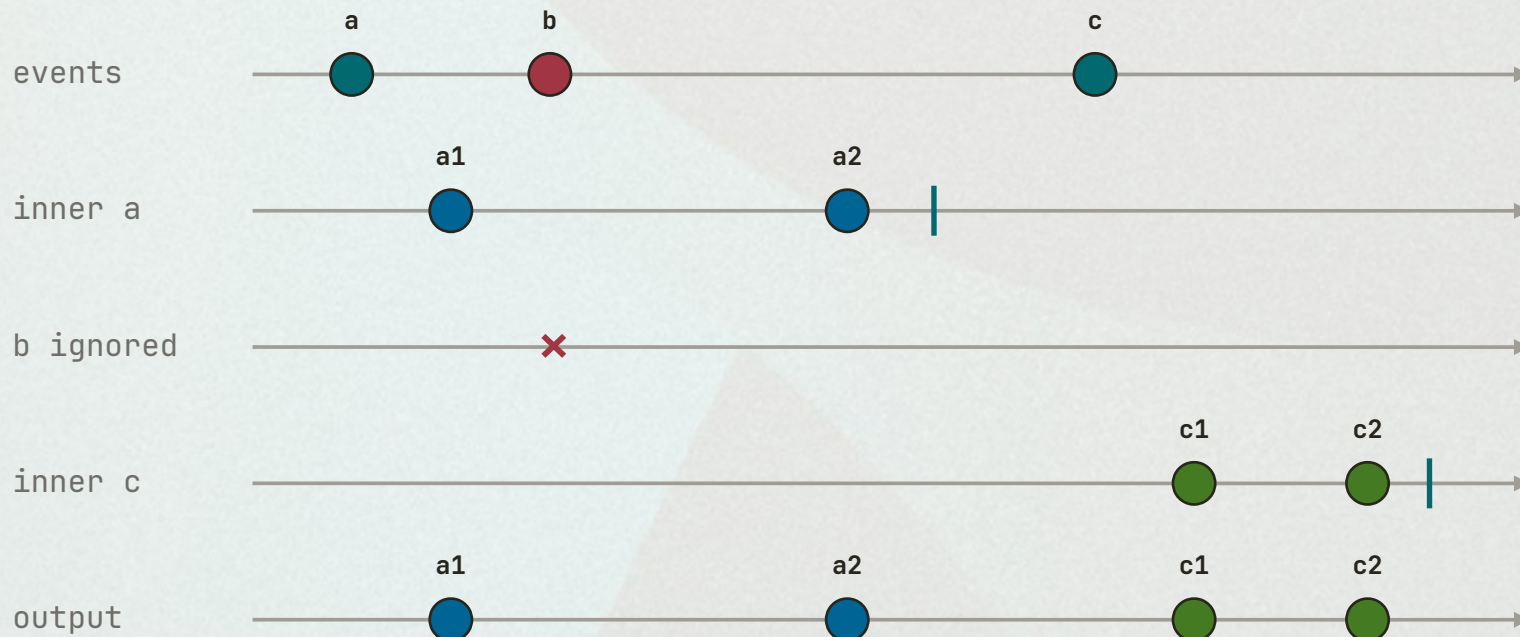
Быстрый input создаёт длинную
очередь и задержку.

ЗАПОМНИ

`concatMap` = «сохрани порядок и ничего не потеряй».

exhaustMap — игнорировать новое

Пока внутренний поток активен, новые события отбрасываются.



Лучше всего

Submit

Login

Защита от двойного клика

Опасно

Пользовательский input может потеряться без ответа.

ЗАПОМНИ

`exhaustMap` = «занят — новые команды не принимаю».

Четыре стратегии на одном экране

Оператор выбирается по продуктовой семантике, а не по привычке.

switchMap

Отменяет прошлое
Главный приоритет: актуальность

mergeMap

Параллелит всё
Главный приоритет: скорость

concatMap

Сохраняет очередь
Главный приоритет: порядок

exhaustMap

Игнорирует новое
Главный приоритет: защита

ЗАПОМНИ

Сначала сформулируйте, что делать с конкурирующей работой. Затем выбирайте оператор.

Алгоритм выбора higher-order оператора

Четыре вопроса приводят к одному из четырёх решений.

01

Новая работа делает прошлую неактуальной?



switchMap

02

Порядок не важен, можно параллельно?



mergeMap

03

Нужно выполнить всё строго по очереди?



concatMap

04

Повторы нужно игнорировать, пока заняты?



exhaustMap

Что теперь умеем

Короткая проверка перед следующим разделом

● Отменять устаревшую работу

● Управлять параллельностью

● Сохранять строгий порядок

● Защищать действия от повторов

Комбинирование и lifecycle

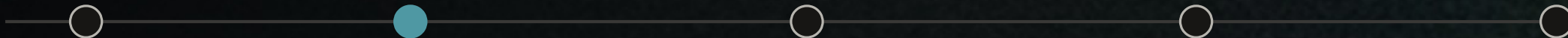
Связать несколько источников и управлять ресурсами

COMBINELATEST

FORKJOIN

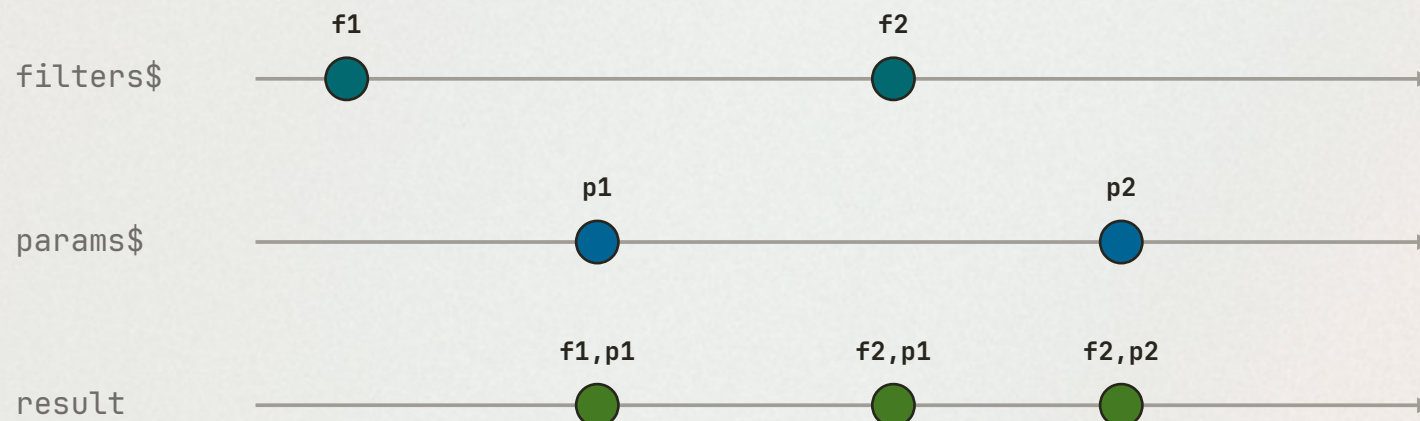
TAKEUNTIL

SHAREREPLAY



combineLatest()

Эмитит последние значения при изменении любого источника.



LIVE SOURCES

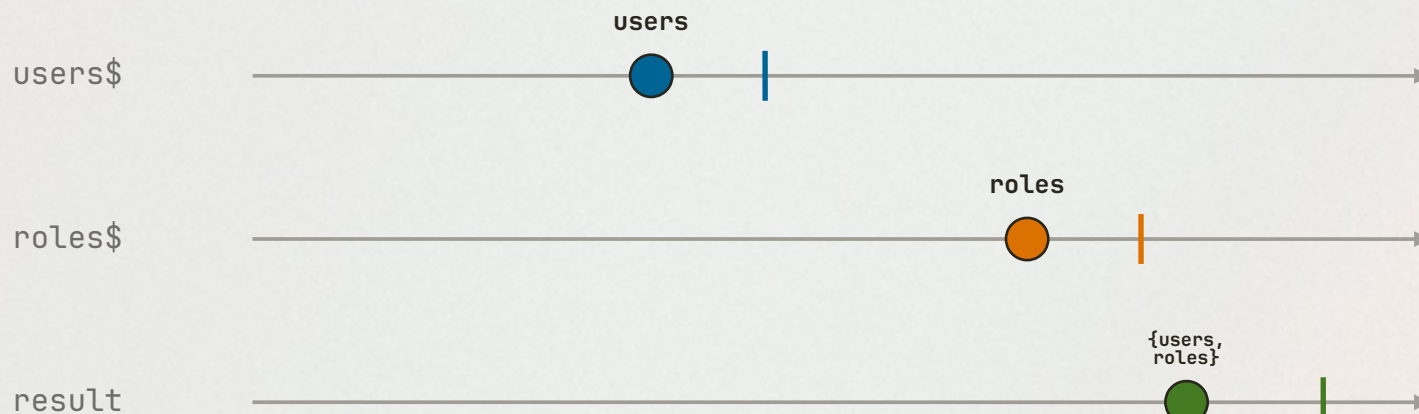
```
combineLatest([
  filters$,
  route.queryParams
])
```

ЗАПОМНИ

Результата нет, пока каждый источник не отправит хотя бы одно значение.

forkJoin()

Ждёт complete всех источников, затем эмитит последние значения.



WAIT FOR ALL

```
forkJoin({  
  users: http.get('/users'),  
  roles: http.get('/roles')  
})
```

ЗАПОМНИ

Бесконечный источник не завершится — `forkJoin` не выдаст результат.

combineLatest vs forkJoin

Выбор зависит от жизненного цикла источников.

combineLatest

Живые потоки
Много результатов
Реагирует на любое изменение
Ждёт первый next каждого

forkJoin

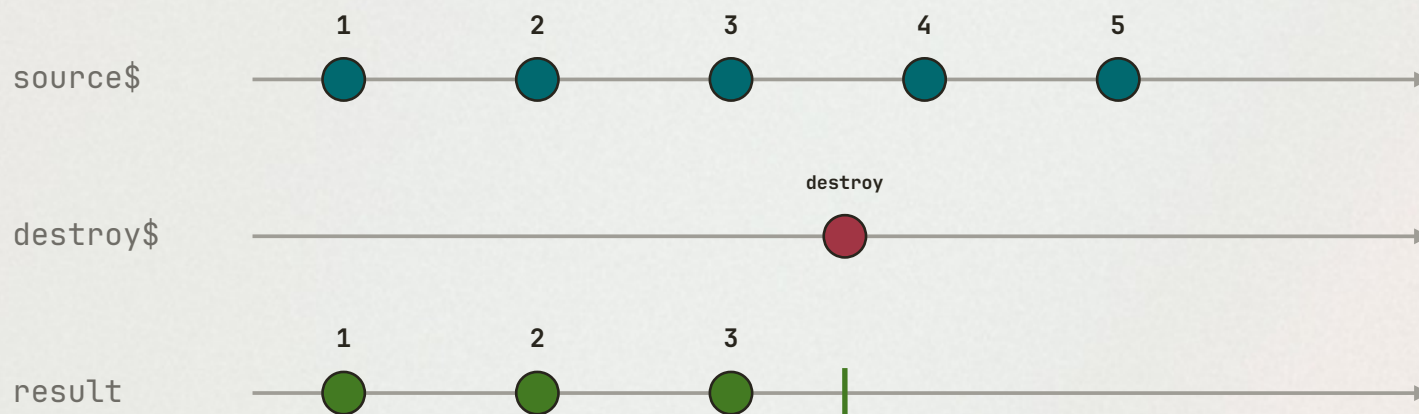
Завершающиеся потоки
Один итоговый результат
Ждёт complete всех
Подходит для группы HTTP

ЗАПОМНИ

Живые значения > combineLatest. Завершающиеся задачи > forkJoin.

takeUntil()

Завершает поток при первом значении управляющего Observable.



BOUNDARY

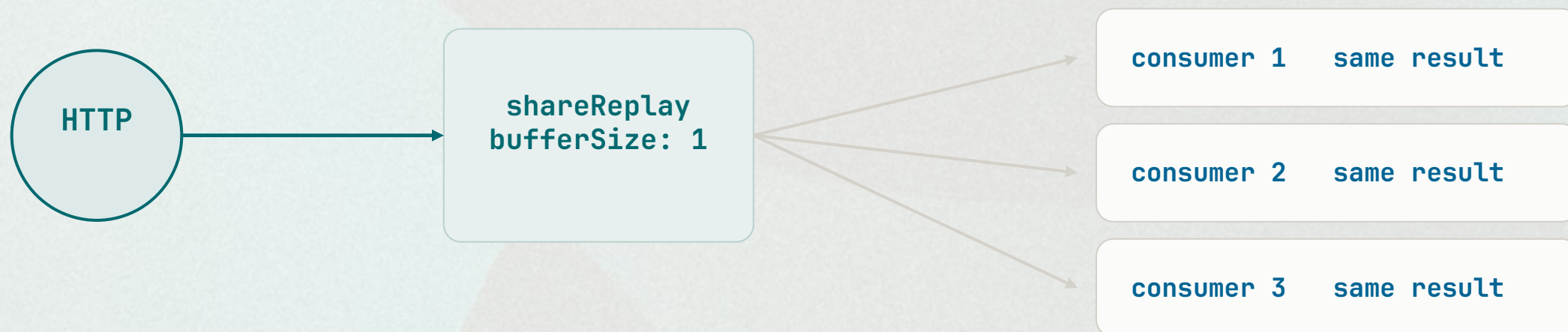
```
source$.pipe(  
  takeUntil(destroy$)  
)
```

ЗАПОМНИ

В Angular 16+ предпочитайте `takeUntilDestroyed()`.

shareReplay()

Одна подписка на источник, последнее значение доступно новым потребителям.

**ЗАПОМНИ**

`'shareReplay'` не вечный кэш: заранее определите `refCount`, сброс и владельца live-источника.

Что теперь умеем

Короткая проверка перед следующим разделом

● Объединять живые источники

● Ждать завершения группы HTTP

● Завершать поток по сигналу

● Делиться результатом осознанно

RxJS в Angular

HttpClient, Router, Forms, шаблоны и lifecycle

HTTP

ROUTER

FORMS

ASYNC



HttpClient возвращает Observable

Один HTTP-вызов обычно эмитит ответ и завершается.

SERVICE

```
getUser(id: number) {  
  return this.http.get<User>(  
    `/api/users/${id}`  
  );  
}
```

Почему это удобно

- Отмена через unsubscribe
- Композиция с Router и Forms
- Единая обработка ошибок

Повтор запроса

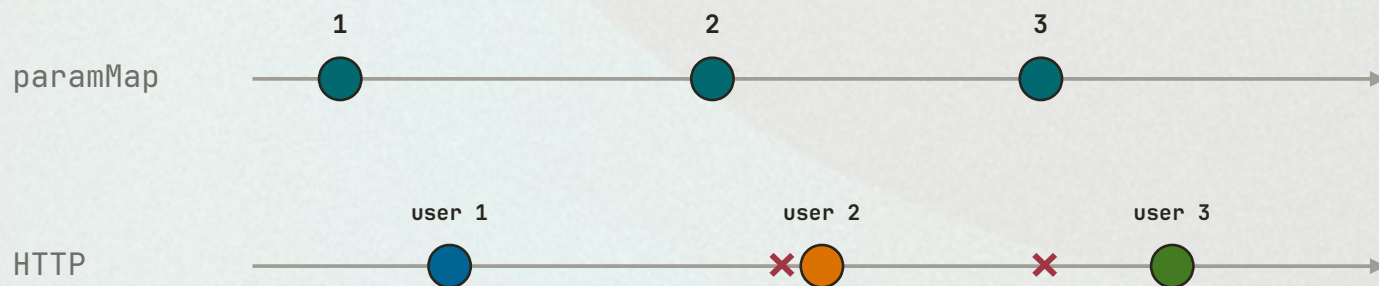
Новая подписка на cold HttpClient Observable создаёт новый запрос.

ЗАПОМНИ

HTTP Observable завершается сам, но композиция и владение подпиской всё равно важны.

Router params › актуальные данные

Каждая навигация создаёт новое значение ID.



ROUTER → HTTP

```
route.paramMap.pipe(  
  switchMap(params =>  
    api.get(params.get('id'))  
  )  
)
```

ЗАПОМНИ

switchMap отменяет устаревший запрос при смене URL.

Reactive Forms как поток

valueChanges превращает ввод пользователя в Observable.

FORM PIPELINE

```
results$ = search.valueChanges.pipe(  
  debounceTime(300),  
  distinctUntilChanged(),  
  switchMap(term ⇒ api.search(term))  
);
```

Смысл цепочки

Пауза › убрать дубли › отменить старый запрос › показать актуальное.

В шаблоне

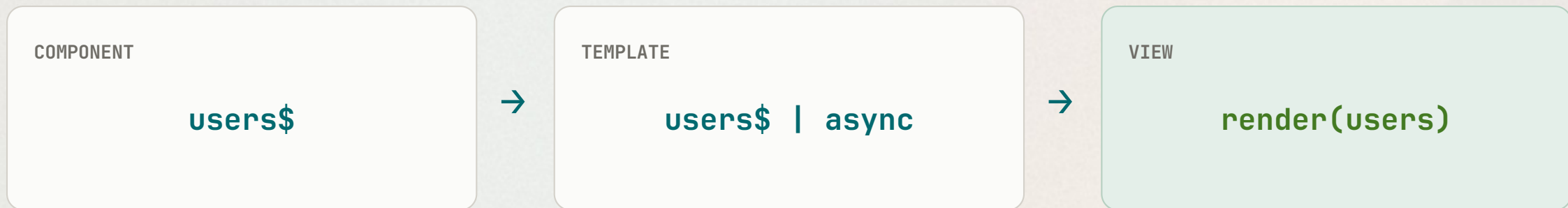
```
results$ | async
```

ЗАПОМНИ

Форма сообщает события; pipeline превращает их в готовое состояние UI.

AsyncPipe управляет подпиской

Подписка создаётся при рендере и очищается при уничтожении view.



TEMPLATE

```
@if (users$ | async; as users) {  
  @for (user of users; track user.id) {  
    <app-user-card [user]="user" />  
  }  
}
```

ЗАПОМНИ

Если значение нужно только шаблону, AsyncPipe обычно проще ручного subscribe.

Очистка подписок в современном Angular

Выберите самый декларативный вариант, который подходит месту использования.

AsyncPipe

Для значений, которые читает шаблон.

takeUntilDestroyed()

Для imperative subscribe внутри компонента.

takeUntil(destroy\$)

Рабочий классический вариант для старых версий.

ЗАПОМНИ

Предпочитайте API, которое связывает ресурс с lifecycle автоматически.

takeUntilDestroyed и injection context

Вызов без аргумента доступен не в любом месте класса.

В ОБЫЧНОМ МЕТОДЕ

```
private destroyRef = inject(DestroyRef);

ngOnInit() {
  this.api.getLive().pipe(
    takeUntilDestroyed(this.destroyRef)
  ).subscribe();
}
```

Без аргумента

Работает в injection context: например, в инициализации поля или конструкторе.

С DestroyRef

Передайте DestroyRef явно в ngOnInit и других обычных методах.

ЗАПОМНИ

Короткий API не отменяет необходимость понимать, кто владеет подпиской.

Что теперь умеем

Короткая проверка перед следующим разделом

- Комбинировать Router и HTTP
- Строить поток из `valueChanges`
- Отдавать подписку `AsyncPipe`
- Использовать `DestroyRef` корректно

Практические паттерны

От продуктовой ситуации к оператору и коду

CHALLENGE

OPERATOR

CODE

MISTAKE



Живой поиск без лишних запросов

Ситуация · потоки · оператор · схема · код · ошибка

Ситуация

Пользователь печатает быстро; запрос на каждый символ создаёт шум.

Потоки

`valueChanges` → строки поиска

Оператор

`debounceTime` +
`distinctUntilChanged` + `switchMap`

`typing` → `pause 300 ms` → `latest term` → `HTTP` → `results`

РЕШЕНИЕ

```
search.valueChanges.pipe(  
  debounceTime(300),  
  distinctUntilChanged(),  
  switchMap(term ⇒ http.get('/api/search', { params: { term } })))  
)
```

ТИПИЧНАЯ ОШИБКА

Не отправляйте запрос на каждый символ и не используйте `mergeMap` для устаревающих результатов.

Отмена устаревшего HTTP-запроса

Ситуация · потоки · оператор · схема · код · ошибка

Ситуация

URL уже изменился, но старый запрос может вернуть данные позже нового.

Потоки

Router paramMap → ID → HTTP

Оператор

switchMap

id:1 —× id:2 —× id:3 —✓

РЕШЕНИЕ

```
route.paramMap.pipe(  
  map(params ⇒ params.get('id')),  
  switchMap(id ⇒ api.getUser(id))  
)
```

ТИПИЧНАЯ ОШИБКА

mergeMap допускает гонку ответов и может показать данные старого маршрута.

Сохранение без повторной отправки

Ситуация · потоки · оператор · схема · код · ошибка

Ситуация

Несколько быстрых кликов создают несколько POST-запросов.

Потоки

clicks → form payload → POST

Оператор

exhaustMap

click ✓ → POST active → click ignored → complete

РЕШЕНИЕ

```
submitClicks$.pipe(  
  exhaustMap(() ⇒  
    http.post('/api/save', form.getRawValue())  
  )  
)
```

ТИПИЧНАЯ ОШИБКА

mergeMap отправит каждый POST; данные могут сохраниться несколько раз.

Загрузка нескольких источников

Ситуация · потоки · оператор · схема · код · ошибка

Ситуация

Экран готов только после получения пользователей и ролей.

Потоки

users HTTP + roles HTTP

Оператор

`forkJoin`

`users complete + roles complete → one combined result`

РЕШЕНИЕ

```
forkJoin({
  users: http.get('/api/users'),
  roles: http.get('/api/roles')
}).subscribe(init);
```

ТИПИЧНАЯ ОШИБКА

`forkJoin` никогда не выдаст результат, если один источник не завершается.

Обработка ошибок без поломки UI

Ситуация · потоки · оператор · схема · код · ошибка

Ситуация

Запрос упал; интерфейс должен показать fallback или ошибку.

Потоки

HTTP → error → replacement Observable

Оператор

`catchError` + `retry` + `finalize`

`request` → `retry` ×2 → `fallback or data` → `finalize`

РЕШЕНИЕ

```
http.get<User[]>('/api/users').pipe(
  retry({ count: 2, delay: 1000 }),
  catchError(() ⇒ of([])),
  finalize(() ⇒ loading = false)
)
```

ТИПИЧНАЯ ОШИБКА

`catchError` обязан вернуть `Observable`; его положение определяет, продолжится ли внешний поток.

Синхронизация фильтров, URL и API

Ситуация · потоки · оператор · схема · код · ошибка

Ситуация

Изменение формы или query params должно обновить одни данные.

Потоки

filters\$ + queryParams → request params

Оператор

combineLatest + switchMap

[latest filters, latest params] → cancel old → API

РЕШЕНИЕ

```
combineLatest([
  filters$.pipe(startWith(initialFilters)),
  route.queryParams
]).pipe(
  switchMap(([filters, params]) ⇒ api.load({ ...filters, ...params }))
)
```

ТИПИЧНАЯ ОШИБКА

Отдельные subscribe создают гонки и рассинхронизацию состояния.

Состояния loading / data / error

Ситуация · потоки · оператор · схема · код · ошибка

Ситуация

Экран должен явно представлять все состояния запроса.

Потоки

trigger → HTTP → discriminated UI state

Оператор

startWith + map + catchError

loading → data | error → retry

РЕШЕНИЕ

```
load$.pipe(  
  switchMap(() ⇒ api.getData().pipe(  
    map(data ⇒ ({ status: 'data', data })),  
    catchError(error ⇒ of({ status: 'error', error })),  
    startWith({ status: 'loading' })  
  ))  
)
```

ТИПИЧНАЯ ОШИБКА

Одна boolean loading не описывает данные, ошибку и повторную загрузку.

Предотвращение утечек

Ситуация · потоки · оператор · схема · код · ошибка

Ситуация

Компонент уничтожен, а long-lived источник продолжает отправлять значения.

Потоки

interval / events / subject → component

Оператор

AsyncPipe или takeUntilDestroyed

component create → subscribe → destroy → unsubscribe

РЕШЕНИЕ

```
private destroyRef = inject(DestroyRef);

ngOnInit() {
  live$.pipe(
    takeUntilDestroyed(this.destroyRef)
  ).subscribe(render);
}
```

ТИПИЧНАЯ ОШИБКА

Ручной массив Subscription работает, но его легко забыть обновить.

Тестирование RxJS-потоков

Ситуация · потоки · оператор · схема · код · ошибка

Ситуация

Реальные таймеры делают тесты медленными и нестабильными.

Потоки

cold marble input → operator → expected output

Оператор

TestScheduler

`-a-b|` → `map(toUpperCase)` → `-A-B|`

РЕШЕНИЕ

```
scheduler.run(({ cold, expectObservable }) => {  
  const source$ = cold('-a-b|');  
  const result$ = source$.pipe(map(v => v.toUpperCase()));  
  expectObservable(result$).toBe('-x-y|', { x: 'A', y: 'B' });  
});
```

ТИПИЧНАЯ ОШИБКА

Не проверяйте только финальное значение: тестируйте отмену, порядок и подписки.

Памятка: ситуация › оператор

Ищите продуктовую семантику, а не знакомое имя.

Пауза ввода

`debounceTime`

Убрать дубли

`distinctUntilChanged`

Отменить старое

`switchMap`

Защитить submit

`exhaustMap`

Сохранить очередь

`concatMap`

Параллелить

`mergeMap`

Ждать все HTTP

`forkJoin`

Объединить live

`combineLatest`

Начальное состояние

`startWith`

Очистить lifecycle

`takeUntilDestroyed`

Четыре вопроса перед выбором

Higher-order operator — это политика конкуренции.

Отменять прошлое?

switchMap

Выполнять параллельно?

mergeMap

Сохранять очередь?

concatMap

Игнорировать повторы?

exhaustMap

ЗАПОМНИ

Сначала выберите поведение при конкуренции. Код оператора станет очевидным.

RxJS за пределами Angular

Операторы работают в любом JavaScript-приложении.

React

Подписка в `useEffect`
`cleanup` вызывает `unsubscribe`
или готовые `observable hooks`

Vue и другие UI

Поток остаётся независимым
адаптер связывает его
с `lifecycle` конкретного UI

ЗАПОМНИ

RxJS — библиотека JavaScript, а не часть Angular.

Источники

Основные материалы для содержания и дальнейшего изучения.

RxJS documentation

<https://rxjs.dev/>

RxJS course

<https://rxjs-course-avy.web.app/lessons>

Angular Challenges

<https://angular-challenges.vercel.app/>

React-RxJS

<https://react-rxjs.org/>

rxjs-hooks

<https://github.com/LeetCode-OpenSource/rxjs-hooks>

Интерфейс — это значения во времени

RxJS даёт язык, чтобы описать их движение, конкуренцию и превращение в состояние UI.



events → stream → operators → UI state